

	Computación	Docente: Ing. Daniel Díaz MSc
	Programación Aplicada	Período Lectivo: 68

			FORMATO DE GUÍA DE PRÁCTICA DE LABORATORIO / TALLERES / CENTROS DE SIMULACIÓN – PARA DOCENTES		
CARRERA: Computación			ASIGNATURA: Programación Aplicada		
NRO. PRÁCTICA:	2	TÍTULO PRÁCTICA: SISTEMA DE GESTION			
OBJETIVO - Crear un programa que muestre los conceptos de interfaces y clases genéricas.					
INSTRUCCIONES:		1. Crear las clases con los atributos y métodos solicitados. 2. Establecer la clase GENERICA correspondiente. 3. Crear la clase de main para la salida. 4. Aplicar todos los conceptos analizados en clase.			
ACTIVIDADES POR DESARROLLAR					
La empresa SportCenter Solutions, dedicada a la administración de espacios recreativos y deportivos, ha solicitado el desarrollo de un sistema de gestión de reservas de canchas deportivas para mejorar el control de sus servicios y optimizar la atención a los usuarios. Actualmente, la empresa realiza el registro de canchas y reservas de manera manual, lo que ha generado diversos inconvenientes, como duplicidad de reservas, dificultad para conocer la disponibilidad real de las canchas, pérdida de información y demora en la atención al cliente. Debido a esta situación, la gerencia considera necesario implementar una aplicación que permita llevar un control ordenado, seguro y eficiente de toda la información relacionada con las canchas disponibles y su uso. La empresa administra distintas canchas deportivas, y de cada una necesita almacenar información básica como un código único, el tipo de cancha, su ubicación y el costo por hora. Además, el sistema deberá permitir conocer si una cancha se encuentra disponible o si ya ha sido reservada por algún usuario. El nuevo sistema deberá permitir registrar nuevas canchas, consultar el listado general de canchas registradas, realizar reservas y también registrar la liberación de una cancha cuando termine su uso. La empresa también desea que el sistema muestre información de control, como la cantidad total de canchas almacenadas y cuántas de ellas se encuentran reservadas en un momento determinado.					

Por políticas internas de infraestructura, la empresa ha establecido que el sistema manejará un máximo de **50 canchas registradas**. También ha indicado que la aplicación debe estar estructurada bajo un enfoque orientado a objetos, utilizando clases independientes para representar las entidades principales del sistema y organizando adecuadamente las operaciones del menú.

Adicionalmente, el departamento técnico de la empresa ha solicitado que la solución sea desarrollada aplicando ciertos conceptos vistos en clase, como el uso de **clases genéricas**, **colecciones dinámicas**, **varargs**, **boxing y unboxing**, y la **separación de cada opción del menú en métodos independientes**, de manera que el programa sea modular, reutilizable y fácil de mantener.

Con base en esta necesidad planteada por la empresa, se solicita **analizar el problema y determinar los requerimientos funcionales y no funcionales del sistema**, así como proponer una solución orientada a objetos que responda a las necesidades descritas.

	Computación	Docente: Ing. Daniel Díaz M S C .
	Programación Aplicada	Período Lectivo: 6587

◆ Opción 3: Registrar préstamo de un libro

El sistema solicitará:

- ISBN del libro a prestar
- Nombre del usuario que solicita el libro

Si el libro **ya está prestado**, el sistema mostrará:

.. "El libro no está disponible. ¿Desea intentar con otro ISBN? (S/N)"

Si el usuario continúa, podrá ingresar nuevamente los datos.

Si el libro **está disponible**, el sistema confirmará:

■ "Préstamo registrado exitosamente."

El estado del libro cambiará a **PRESTADO**, y se guardará el nombre del usuario.

◆ Opción 4: Registrar devolución de un libro

El sistema pedirá el **ISBN** del libro devuelto.

Si el libro existe y se encuentra prestado, se mostrará:

■ "Devolución registrada. El libro ahora está disponible."

Si el ISBN no existe o el libro ya estaba disponible, el sistema informará del error correspondiente y volverá al menú.

◆ Opción 0: Salir del sistema

Antes de salir, el sistema confirmará la acción:

.. "¿Está seguro de que desea salir? (S/N)"

Si la respuesta es afirmativa, la aplicación se cerrará.

De lo contrario, regresará al menú principal.

⚙ Requerimientos técnicos y de diseño

El programa debe incluir al menos **dos clases principales**:

1. **Clase Libro**
Representa los datos y el estado de cada ejemplar.
2. **Clase Biblioteca**
Contiene la colección de libros y el método main() que gestiona la aplicación.

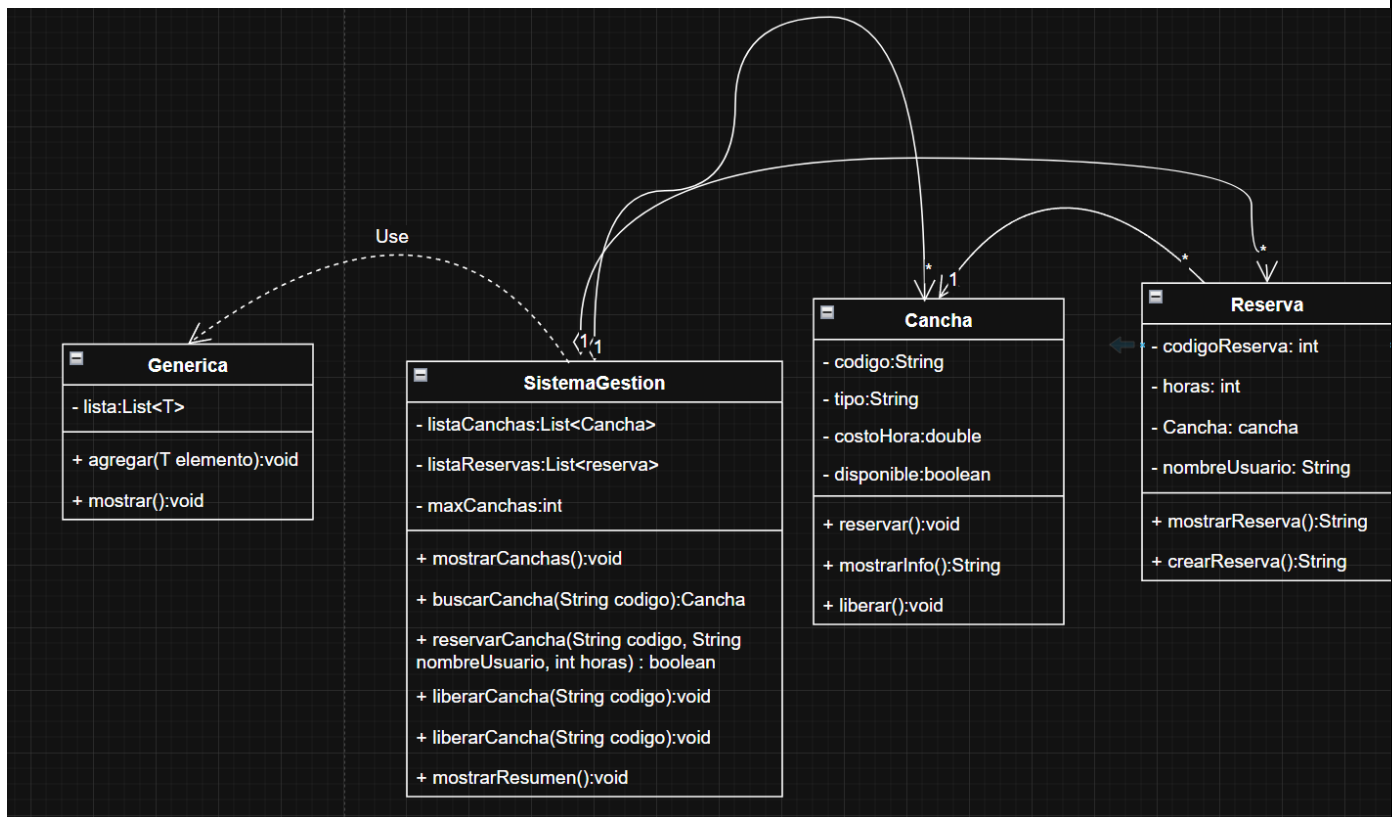
Requerimiento funcionales:

1. Inicio de sesión
2. Registrar canchas deportivas
El sistema debe permitir ingresar nuevas canchas con datos como código, tipo, ubicación y costo por hora.
3. Consultar listado de canchas
El usuario debe poder ver todas las canchas registradas en el sistema.
4. Ver disponibilidad de canchas
El sistema debe indicar si una cancha está disponible o reservada.
5. Realizar reservas de canchas y liberación de canchas
El usuario podrá reservar una cancha disponible.

⚡ Requerimientos no funcionales

1. Límite de capacidad
El sistema debe manejar un máximo de 50 canchas registradas.
2. Estructura orientada a objetos
Debe estar desarrollado usando clases que representen las entidades del sistema.
3. Uso de clases genéricas
Para permitir una gestión flexible de datos.
4. Uso de boxing y unboxing
Para manejar correctamente tipos primitivos en el sistema.
5. Modularidad del sistema
Cada opción del menú debe estar separada en métodos independientes.
6. Seguridad del sistema (disponibilidad, integridad, autenticación, confidencialidad)
7. Eficiencia en el funcionamiento
8. Implementar una demo en modo texto

Diagrama UML:



El sistema debe aplicar los siguientes conceptos:

- **Clases genéricas:** para una gestión flexible de colecciones (por ejemplo, `generic<T,S>`).
- **Uso de args (varargs):** para manejar entradas variables de datos o mensajes.

- **Boxing y unboxing:** en el manejo de tipos primitivos (como contadores o estados).
- **Colecciones dinámicas (List)** en lugar de arreglos estáticos.
- Separar cada opción del menú en métodos funcionales

RESULTADO(S) OBTENIDO(S):

El estudiante podrá aplicar los conceptos de Herencia, interfaces y clases genéricas. en los casos que crea conveniente con el fin de organizar, reutilizar código y reducir en gran medida la redundancia en varias clases.

CONCLUSIONES:

- El estudiante debe plantear al menos dos conclusiones.

RECOMENDACIONES:

- El estudiante debe plantear al menos dos recomendaciones.

Docente / Técnico Docente:

Firma: _____

 UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR	Computación	Docente: Ing. Daniel Díaz MSc
	Programación Aplicada	Período Lectivo: 57